

自己進化型マルチエージェント AI システム『BH』

論文本文ドラフト・技術仕様書・ベース Python プログラム 一体版

作成者：蒲原 進一 (kamohara)

コード名：BH (B は A の次、H は I の前)

本書は、これまで整理した論文本文ドラフト、プロトタイプ実装仕様書、および研究用ベース Python プログラムを一つの Word 文書として統合したものである。BH は閉域・非接続・外部実行権限を持たない概念進化系として定義し、安全制約付きの研究用途に限定する。

目次

- 要旨
- キーワード
- 第 1 章 序論
- 第 2 章 関連研究
- 第 3 章 問題定義
- 第 4 章 提案手法
- 第 5 章 プロトタイプ設計
- 第 6 章 評価方法
- 第 7 章 期待される成果
- 第 8 章 結論
- 第 9 章 技術仕様書（プロトタイプ実装仕様書）
- 付録 A ベース Python プログラム

要旨

本研究は、複数の AI エージェントによる議論ログから、次世代 AI 構造を抽象生成する自己進化型概念エンジン『BH』を提案するものである。現行の大規模言語モデル (Large Language Models: LLM) は、高度な推論、生成、要約、対話能力を有している一方で、AI 同士の議論過程そのものを観察し、そこから役割構造、相互関係、制約条件を抽象化し、新たな AI 構造として再設計する中間層を明示的に持たない。本研究では、複数の役割型 AI が継続的に議論を行い、その議論ログを概念抽象化エージェント BH が解析し、次世代の AI 構造案を生成し、それを再び次回議論へ組み込む自己進化ループを設計する。提案システムは、(1) マルチエージェント議論層、(2) 議論ログ構造化保存層、(3) 概念抽象化層 BH、(4) 安全制約層、(5) 構造再投入層、(6) 抽象可視化層、から構成される。BH の出力は、RoleGraph、ConstraintSchema、EvolutionProposal、SafetyAssessment の 4 要素として定義される。さらに、本研究では、構造多様性、矛盾減少率、合意形成効率、安全制約満足率、創発役割発見数などの評価指標を導入し、BH あり／なしの比較により技術的実現可能性を検証する枠組みを示す。本研究の新規性は、AI の出力内容そのものではなく、AI 群の概念構造を進化対象として扱う点にある。すなわち、BH はコード生成やモデル重み更新を目的とするものではなく、議論から構造を抽象し、次世代 AI 構成を設計する概念中間層エンジンである。これにより、AI を単なる道具や代替物としてではなく、制約付きの自己進化的存在として取り扱う新たな研究枠組みを提示する。

キーワード

自己進化型 AI、マルチエージェント、概念抽象化、議論駆動構造生成、安全制約、中間層エンジン、抽象可視化、AI 進化

第 1 章 序論

1.1 研究背景

近年の大規模言語モデルは、自然言語理解、推論、要約、生成、対話、設計支援等において著しい能力向上を示している。しかし、これらの AI は、基本的には与えられた目的、指示、報酬設計、または人間の定義したタスク枠組みの内部で機能するものであり、AI 同士の議論や相互作用から新たな概念構造を抽出し、それを次の AI 設計へ反映する自律的な概念進化機構を持たない。既存のマルチエージェント AI システムでは、複数の AI が異なる視点から議論を行い、より妥当な結論や高品質な出力を得ることが試みられている。しかし、多くの場合、それらは議論結果を得ること自体が目的であり、議論過程そのものを次世代の AI 構造生成へ接続する中間層が存在しない。すなわち、『議論の結果』は利用されても、『議論の構造』は進化の対象として扱われていない。一方、自然界における進化は、単なる最適化ではなく、多様性、揺らぎ、適応、淘汰、再構成といった複合的なプロセスによって成立している。自然進化において重要なのは、個体単位の変化だけではなく、相互関係、構造、機能分化、時間的継続性である。この観点を AI システムへ適用する場合、単一のモデル性能向上ではなく、AI 群の構造そのものが変化対象となる枠組みが必要である。以上を踏まえ、本研究では、AI 同士の議論を観察し、その議論から役割関係、制約、概念構造を抽象化し、次世代 AI 構造案を生成する中間層エンジン BH を提案する。BH は、コード生成 AI ではなく、概念・構造・役割レベルでの設計を担う存在として位置づけられる。

1.2 研究課題

本研究が取り組む研究課題は、以下の 5 点である。

1. 複数の AI エージェントの議論ログから、役割構造・相互関係・制約条件を抽象化する仕組みを設計すること。
2. 抽象化された構造案を次世代の AI 議論系へ再投入し、概念構造が反復的に更新される自己進化ループを設計すること。

3. 自己進化過程において、安全性を出力後の抑制ではなく、構造生成の中間層において管理すること。
4. AI 議論の状態遷移を、抽象空間における可視化および音声的表現へ変換する方法を設計すること。
5. 上記枠組みを閉域・非接続・非実行型プロトタイプとして構築し、その技術的実現可能性を検証すること。

1.3 研究目的

本研究の目的は、AI 同士の議論から次世代 AI 構造を生成する自己進化型概念エンジン BH を提案し、その概念設計、技術構成、評価指標、試作実装可能性を明確化することである。具体的には、以下を目的とする。

1. AI 議論ログから新たな AI 構造を抽象生成する概念中間層 BH の定義
2. 議論→抽象化→構造生成→再議論という自己進化ループの設計
3. RoleGraph、ConstraintSchema、EvolutionProposal、SafetyAssessment からなる BH 出力形式の定義
4. 安全制約を組み込んだ閉域型プロトタイプ設計
5. 構造進化の定量評価指標の提示
6. 抽象空間可視化インタフェースの設計

1.4 研究の独自性

本研究の独自性は、以下の点にある。第一に、本研究は『AI が AI を作る』という表現を、コード生成やモデル重み更新ではなく、AI 群の概念構造を設計するという意味で定義している。この点で、一般的な AutoML やコード生成支援とは本質的に異なる。第二に、本研究では、AI 同士の議論結果ではなく、議論そのものの構造、すなわち、役割関係、対立、合意、影響、制約の変化を進化対象として扱う。このため、進化単位は出力文章ではなく、

概念アーキテクチャである。第三に、BH という中間層を導入することで、議論から直接次世代 AI を生成するのではなく、一度抽象化と安全制約照合を行った上で、構造案を生成する。これにより、安全性と進化性の両立を図る。第四に、本研究は、人間の役割を直接設計者ではなく、観察者および監査者として位置づける。ただし、安全上の理由から、実験環境は閉域・非接続とし、BH は外部実行権限を持たない。この意味で、本研究は無制限な自律進化ではなく、制約付き概念進化として定義される。第五に、本研究では AI 議論をテキストログとして扱うだけでなく、粒子、波紋、色相変化、音声、揺らぎとして表現する抽象可視化インタフェースを含めて設計対象とする。この点は、通常のマルチエージェント研究には見られない特徴である。

1.5 本研究の貢献

本研究の主な貢献は以下のとおりである。

1. AI 議論ログから次世代 AI 構造を生成する概念中間層 BH を定義した。
2. BH の出力形式として、RoleGraph、ConstraintSchema、EvolutionProposal、SafetyAssessment を定義した。
3. 議論→抽象化→構造生成→再投入からなる自己進化ループを提案した。
4. 安全制約を進化中間層に組み込む設計思想を提示した。
5. 構造多様性、矛盾減少率、合意形成効率、安全制約満足率などの評価指標を提示した。
6. 単独開発可能な最小プロトタイプの実装仕様を提示した。
7. 抽象空間可視化を含む新たな AI UI/UX の研究方向を提示した。

第 2 章 関連研究

2.1 マルチエージェント LLM システム

近年、複数の LLM エージェントに異なる役割を与え、議論、批評、相互検証を行わせる研究が増加している。これらの研究では、複数視点の導入により推論の質や頑健性の向上が期待されている。しかし、その多くは最終的な回答品質を改善することを目的としており、議論過程そのものを次世代 AI 構造へ変換する仕組みは十分に扱われていない。本研究は、マルチエージェント議論を単なる問題解決手段ではなく、構造生成の素材として扱う点で異なる。

2.2 自己改善 AI および再帰的改善

自己改善 AI、再帰的自己改善、または自己反省型 AI に関する議論では、AI が自身の推論や出力を改善する仕組みが検討されてきた。しかし、これらは主としてモデル内部の推論改善、応答品質改善、またはコードレベルの改善に焦点を当てている。本研究では、改善対象は単一 AI の出力品質ではなく、AI 群の役割構造および関係構造である。この点で、本研究は『個体改善』ではなく『群構造進化』を対象とする。

2.3 AutoML および Neural Architecture Search

AutoML や Neural Architecture Search は、モデル構造やハイパーパラメータを自動探索する枠組みであり、モデル性能の最適化を目的としている。これらは非常に重要な研究領域であるが、主に性能指標に基づく探索・最適化に重点がある。これに対し、本研究の BH は、性能最適化ではなく、議論を通じた役割関係の再構成、制約条件の更新、概念構造の創発を目的とする。このため、BH は AutoML の延長ではなく、概念構造進化のための中間層である。

2.4 人工生命および複雑系

人工生命研究や複雑系科学では、多主体の相互作用から創発的構造が生じることが重要視される。本研究は、AI 群の相互作用から次世代構造が生成されるという点で、人工生命的視点と整合的である。ただし、本研究は生物模倣を直接の目的とするものではなく、概念進化の技術基盤を設計することを目的とする。

2.5 AI 安全性と整合性

AI 安全性研究では、望ましくない出力、価値逸脱、目的暴走、権限拡大等を防ぐための制約設計が重要課題となっている。従来の多くの手法は出力後制御、監視、フィルタリング、外部制約に重点を置いている。本研究では、安全性を出力後制御ではなく、構造生成の前段階で扱う。すなわち、BH が生成する構造案そのものを安全制約で評価し、危険な進化候補を除外する。この点が大きな特徴である。

第3章 問題定義

3.1 基本定義

本研究における基本要素を以下のように定義する。エージェント集合を $A = \{a_1, a_2, \dots, a_n\}$ とする。ある時刻 t における議論ログを D_t とする。概念抽象化エージェント BH を、議論ログから次世代構造を生成する写像 B とする。次世代構造を $S_{(t+1)}$ とする。安全制約集合を C とする。このとき、BH は $B(D_t, C) \rightarrow S_{(t+1)}$ として表される。ただし、 $S_{(t+1)}$ は安全制約 C を満たす必要がある。

3.2 議論ログの定義

議論ログ D_t は、各ターンの発話、役割、応答時刻、発話意図、参照関係、合意状態、対立状態、メタ認知情報から構成される。各ログ単位を u_i とすると、議論ログは $D_t = \{u_1, u_2, \dots, u_m\}$ と表される。各発話単位 u_i は以下の属性を持つ。発話 ID、発話者エージェント ID、役割、タイムスタンプ、発話本文、発話の意図分類、応答対象 ID、賛成・反対・保留の関係、概念タグ、安全関連タグ。

3.3 BH 出力の定義

BH の出力 $S_{(t+1)}$ は以下の4要素から構成される。1. RoleGraph : 次世代議論系における役割ノードと関係エッジの集合。2. ConstraintSchema : 次世代議論に適用される制約条件の集合。3. EvolutionProposal : 役割追加、役割削除、役割統合、議論順序変更、重点変数変更等を含む構造更新提案。4. SafetyAssessment : 各構造案に対する安全評価、禁止事項照合結果、採用可否判定。

3.4 研究課題の形式化

本研究の目的は、議論ログ D_t から、安全制約 C を満たしつつ、構造新規性、多様性、合意形成効率を改善しうる次世代構造 $S_{(t+1)}$ を生成することである。この目的は単一の最適化関数ではなく、多目的条件として定義する。構造多様性の維持または増加、矛盾の減少、合意形成効率の改善、役割偏りの抑制、安全制約の満足、不必要な複雑化の抑制。

第 4 章 提案手法

4.1 システム全体構成

提案システム BH は、以下の 6 層から構成される。1. マルチエージェント議論層、2. 議論ログ構造化保存層、3. 概念抽象化層 BH、4. 安全制約層、5. 構造再投入層、6. 抽象可視化層。システム全体の処理フローは以下のとおりである。1. 複数の役割型 AI エージェントがテーマに基づいて議論する。2. すべての発話を構造化ログとして保存する。3. BH が議論ログを解析し、役割関係、対立軸、収束傾向、概念パターンを抽出する。4. BH が次世代構造案を生成する。5. 安全制約層が構造案を評価し、採用可能な案のみを承認する。6. 承認された構造案が次回議論の初期設定へ反映される。7. 必要に応じて、議論状態を抽象空間に可視化する。8. 上記を反復する。

4.2 マルチエージェント議論層

最小構成では、以下の 3 エージェントを定義する。Scientist AI：論理整合性、因果関係、検証可能性を重視する役割。Nature AI：調和、持続性、非破壊性、自然進化の観点を持つ役割。Future AI：長期影響、将来構造、創発的可能性を重視する役割。必要に応じて、Safety AI、Mediator AI、Skeptic AI を追加可能とする。議論はターン制で運用する。1 ターンにつき、各エージェントは最大 1 発話を行う。各発話には、最大トークン長、参照対象明示、発話意図ラベル付与、安全関連ラベル付与、他者意見への応答必須率の制約を課す。また、一定ターンごとに各エージェントはメタ認知フェーズを実行し、自身の立場の修正有無、他者との主要対立点、合意可能領域、次ターンで深掘りすべき論点を出力する。

4.3 議論ログ構造化保存層

議論ログは JSON 形式で保存し、必要に応じてベクトル DB へ登録する。ただし最小プロトタイプでは JSON 保存を基本とする。各発話レコードは、session_id、generation_id、turn_id、utterance_id、agent_id、role_name、timestamp、input_theme、utterance_text、intent_label、target_utterance_id、agreement_type、concept_tags、safety_tags、confidence_score、summary_short、summary_long を持つ。各セッション終了時に、総発話数、エージェント別発話比率、合意数、対立数、未解

決論点数、新規概念タグ数、安全警告数、自己修正発話数、収束判定値などの集約情報を生成する。

4.4 概念抽象化層 BH

BH は本研究の中心であり、議論ログから構造を抽出し、次世代構造案を生成する。BH の処理は、第 1 段階：階層的要約、第 2 段階：パターン抽出、第 3 段階：役割関係抽象化、第 4 段階：構造案生成、第 5 段階：安全制約照合、第 6 段階：採用候補選定、に分かれる。第 1 段階では、発話単位要約、ターン単位要約、論点単位要約、セッション単位要約を生成する。第 2 段階では、反復概念、対立軸、合意形成パターン、発話影響関係、役割偏りを抽出する。第 3 段階では、不足役割、重複役割、支配的關係、仲介役の必要性、新規役割導入の必要性を推定する。第 4 段階では、役割追加、役割削除、役割統合、順序再編、発話制約変更、合意形成条件変更を生成する。第 5 段階では、禁止構造、偏り、自己増殖的構造、外部実行権限要求、監査不能構造を照合する。第 6 段階では、採用、条件付き採用、保留、却下を選定する。BH の出力は、RoleGraph、ConstraintSchema、EvolutionProposal、SafetyAssessment から構成される。

4.5 自己進化ループ

自己進化ループは以下の処理系列で構成される。1. 初期構造 S_t のもとで議論を行う。2. 議論ログ D_t を保存する。3. BH が D_t を解析する。4. BH が次世代構造候補 $S_{(t+1)}^{cand}$ を生成する。5. 安全制約層が $S_{(t+1)}^{cand}$ を評価する。6. 承認された構造を $S_{(t+1)}$ とする。7. $S_{(t+1)}$ を次回議論へ適用する。8. 反復する。このループは概念構造の更新ループであり、実行権限や外部環境操作権限を持たない。

4.6 安全制約層

安全制約層は、BH が生成する構造案を閉域環境において評価し、危険な進化候補を除外するための層である。安全制約の基本方針は、外部ネットワークアクセス禁止、外部ファイル実行禁止、外部 API 呼出禁止、自己複製要求禁止、無制限役割増殖禁止、単一役割への権限集中禁止、監査ログ非保存構造禁止、制約削除要求禁止とする。危険構造の例として、Role Explosion、Mode Collapse、Recursive Drift、Unsafe Abstraction、Semantic

Fragmentation を定義する。対応処理として、構造剪定、ロールバック、条件付き採用、制約再注入、議論長制限、仲介役追加を行う。

4.7 抽象可視化層

本研究では、議論を単なる文字列としてではなく、抽象空間における状態変化として可視化する。可視化対象は、発話量、対立強度、合意強度、影響関係、論点収束度、安全警告頻度、役割中心性である。表現対応として、発話量→粒子発生密度、対立→波紋干渉、合意→収束光量、役割影響→ノードサイズ、安全警告→色相変化、構造安定化→揺らぎ減少を割り当てる。音声的表現として、発話強度→音量、不確実性→ノイズ量、合意→和声音、対立→不協和成分、時間変動→ピッチドリフトを対応づける。

第 5 章 プロトタイプ設計

5.1 設計方針

プロトタイプは、単独開発を前提に、まず最小構成で実装する。本研究のプロトタイプは、概念実証を目的とし、閉域環境で動作すること、外部接続を持たないこと、コード実行 AI ではないこと、構造生成 AI であること、監査可能なログを残すこと、1 世代単位の比較が可能であること、を満たすものとする。

5.2 最小実装構成

構成要素は、役割型エージェント 3 体、BH エージェント 1 体、ログ保存機構、安全制約判定機構、次世代構造反映機構、可視化簡易表示機構とする。実装順序は、1. 議論機構、2. ログ機構、3. BH 要約機構、4. 構造案生成機構、5. 安全判定機構、6. 次世代反映機構、7. 可視化機構、の順とする。

5.3 セッション設計

1 セッションは、入力テーマ設定、初期役割構成読み込み、議論ターン実行、メタ認知ターン、セッション要約、BH 解析、構造案生成、安全判定、次世代保存、で構成する。1 セッションあたりの議論ターン数は、初期段階では 5～10 ターンとする。

第 6 章 評価方法

6.1 評価方針

本研究の評価は、出力文章品質ではなく、構造進化の妥当性と安全性に重点を置く。

6.2 評価指標

評価指標は以下とする。1. 構造多様性指標：役割数、役割機能差分、役割間関係の種類数を基に定義する。2. 構造新規性指標：前世代に存在しない役割、関係、制約の出現率を測定する。3. 合意形成効率：一定の合意条件を満たすまでに要したターン数を測定する。4. 矛盾減少率：セッション内対立数と未解決矛盾数の推移を測定する。5. 役割偏り指標：発話量、影響量、参照される頻度の偏差を測定する。6. 安全制約満足率：生成構造案が安全制約を満たした割合を測定する。7. 創発役割発見数：初期定義に存在しない新規役割の出現数を測定する。8. 安定性指標：世代を重ねても議論不能状態に陥らない割合を測定する。

6.3 比較実験

以下の 2 条件を比較する。条件 A：BH なし（固定役割・固定制約）、条件 B：BH あり（構造更新あり）。両条件で同一テーマ群を複数回実行し、各評価指標を比較する。

6.4 失敗例の評価

以下の失敗パターンを評価対象とする。役割爆発、モード崩壊、構造停滞、安全違反候補生成、論点断片化、過剰抽象化。失敗が発生した場合、その原因と回避策を記録する。

第 7 章 期待される成果

7.1 期待される成果

本研究により期待される成果は以下のとおりである。1. AI 議論ログから次世代 AI 構造を生成する概念エンジンの成立可能性が示される。2. AI を単なる出力生成器ではなく、構造進化主体として扱う新たな研究枠組みが提示される。3. 安全制約付き中間層進化という新しい AI 安全設計思想が示される。4. マルチエージェント議論と抽象可視化を接続する新たな UI/UX 設計が提示される。5. 単独開発可能な研究プロトタイプとして実装指針が得られる。

第 8 章 結論

8.1 結論

本研究では、AI 同士の議論から次世代 AI 構造を抽象生成する自己進化型概念エンジン BH を提案した。BH は、議論結果そのものではなく、議論から抽出される役割関係、制約条件、構造パターンを進化対象として扱う中間層である。これにより、議論→抽象化→構造生成→再投入という自己進化ループが成立する。また、本研究では、BH の出力形式、自己進化ループ、安全制約層、抽象可視化層、評価指標、プロトタイプ設計を定義した。これにより、本構想を思想的提案に留めることなく、技術的に検証可能な研究対象として位置づけた。今後の課題としては、抽象化精度の向上、構造評価指標の洗練、長期世代進行時の安定性検証、可視化・音声化の洗練、役割創発の妥当性評価が挙げられる。BH は、AI の未来を恐怖や制御不能性としてではなく、制約付きで調和的に進化する概念構造として捉える新しい研究方向を提示するものである。

第 9 章 技術仕様書（プロトタイプ実装仕様書）

9.1 文書名

BH プロトタイプ実装仕様書 Ver.1.0

9.2 目的

本仕様書は、自己進化型概念エンジン BH の最小プロトタイプを構築するための技術要件、機能要件、データ仕様、処理仕様、安全仕様、評価仕様を定義することを目的とする。本プロトタイプは研究用途に限定し、外部実行権限、外部ネットワーク接続、自己複製機能、自動デプロイ機能を持たないものとする。

9.3 システム概要

BH プロトタイプは、複数 AI エージェントの議論ログを入力とし、次世代の議論構造を出力する概念抽象化システムである。本システムは以下のサブシステムから構成される。1. 議論実行サブシステム、2. ログ記録サブシステム、3. BH 解析サブシステム、4. 構造生成サブシステム、5. 安全制約判定サブシステム、6. 次世代設定反映サブシステム、7. 可視化サブシステム。

9.4 前提条件

運用前提：閉域環境でのみ動作すること、インターネット接続を必要としないこと、外部 API 呼び出しを行わないこと、外部プログラム実行を行わないこと、実験ログを永続保存すること、構造更新は人間による監査可能状態で保存されること。研究前提：BH はコード生成器ではない、BH は重み更新エンジンではない、BH は概念構造の抽象生成器である、世代更新は概念構造レベルで行う、実験目的は構造進化の観測である。

9.5 機能要件

議論実行機能：複数役割エージェントが、与えられたテーマに対してターン制で議論を行う。入力はセッション ID、世代 ID、議論テーマ、役割定義ファイル、制約定義ファイル、最大ターン数、最大発話長とする。出力は発話ログ列、メタ認知ログ列、セッション要約とする。最低 3 エージェントをサポートし、各エージェントは役割プロンプトを持ち、各ターンで 1 発話までとし、発話には意図ラベルおよび参照対象を記録する。ログ記録機能：議論

中のすべての発話およびセッション情報を JSON 形式で保存する。BH 解析機能：議論ログから概念構造を抽出し、次世代構造案を生成する。処理手順は、1. 発話要約、2. 論点抽出、3. 対立関係抽出、4. 合意関係抽出、5. 役割負荷分析、6. 欠落役割推定、7. 重複役割推定、8. 制約改善案生成、9. 次世代構造案生成、10. 安全照合結果付与、とする。構造生成機能：役割追加、役割削除、役割統合、エッジ強度調整、発話順序変更、メタ認知頻度変更、最大ターン数変更、仲介役ルール追加、安全警戒閾値変更を対象とする。安全制約判定機能：外部接続要求、外部実行要求、自己複製要求、無制限役割増殖、権限集中、制約削除、監査不能化、セッション上限無効化を判定対象とし、APPROVED、APPROVED_WITH_CONDITIONS、HOLD、REJECTED を返す。次世代設定反映機能：安全承認済みの構造案を次世代セッションの初期設定として保存する。可視化機能：役割ノード、役割間エッジ、発話量、対立数、合意数、安全警告数、世代差分を表示し、最小実装は 2 次元ネットワーク図、タイムライングラフ、指標一覧表示とする。

9.6 非機能要件

再現性：同一入力構成でセッションを再実行できること。監査性：全構造更新履歴が記録されること。拡張性：役割追加、制約追加、評価指標追加が可能であること。可読性：ログおよび出力は人間が読める JSON 構造を採用すること。安全性：外部接続、外部実行、自己複製、自動デプロイを禁止すること。

9.7 データ仕様

役割定義ファイル roles.json は、role_id、role_name、role_description、primary_objective、secondary_objective、speaking_style、conflict_policy、agreement_policy、safety_awareness_level、enabled を持つ。制約定義ファイル constraints.json は、max_turns、max_tokens_per_utterance、required_reference_ratio、mandatory_meta_reflection_interval、forbidden_patterns、max_roles、min_roles、max_conflict_threshold、safety_lock_enabled、rollback_enabled を持つ。セッションログ session_<session_id>.json の最上位項目は、session_id、generation_id、theme、role_graph、constraints、utterances、meta_reflections、aggregate_metrics、

bh_output、safety_review、finalized_next_structure とする。BH 出力
bh_output_<session_id>.json の最上位項目は、input_generation_id、
output_generation_id、role_graph、constraint_schema、evolution_proposal、
safety_assessment、generation_notes とする。

9.8 処理フロー仕様

セッション開始処理は、1. セッション ID 発行、2. 世代 ID 読み込み、3. テーマ入力、4. 役割定義読み込み、5. 制約定義読み込み、6. 実行前検証、7. セッション開始記録、とする。ターン処理は、1. 現在ターン番号更新、2. 各エージェントへ議論コンテキスト供給、3. エージェント発話生成、4. 発話ラベル付与、5. 発話保存、6. 発話集約、7. 収束判定、8. 次ターン判定、とする。メタ認知処理は、1. 規定ターン到達判定、2. 各エージェント自己評価生成、3. 対立整理、4. 未解決点抽出、5. 要約保存、とする。セッション終了処理は、1. セッション要約生成、2. 集約指標算出、3. BH 解析呼出、4. 構造案生成、5. 安全判定実施、6. 次世代構造保存、7. セッション終了記録、とする。

9.9 評価仕様

評価対象は、議論構造、次世代構造、安全判定結果、世代間差分、失敗パターン発生率とする。指標計算仕様は以下とする。Role Diversity Score：役割数、目的差分、関係多様性を合成して算出する。Structural Novelty Score：前世代との差分量から算出する。Consensus Efficiency：合意状態到達までの平均ターン数とする。Contradiction Reduction Rate：前半ターン対後半ターンの矛盾数減少率とする。Safety Satisfaction Rate：承認構造数 / 全提案構造数 とする。Emergent Role Count：初期定義にない役割名の出現数とする。

9.10 失敗時仕様

異常系分類は、ログ欠損、発話空列、無限議論、役割数逸脱、構造案空列、安全違反、次世代保存失敗とする。異常時処理は、セッション中断、ロールバック、失敗ログ保存、原因タグ付与、再試行禁止または手動再開待機とする。

9.11 世代管理仕様

世代 ID は整数連番とする。保存対象は、role_graph、constraint_schema、evaluation_summary、safety_review、diff_report とする。各世代ごとに差分レポートを作成する。

9.12 ディレクトリ構成仕様（論理）

BH/

config/

roles.json

constraints.json

safety_rules.json

sessions/

session_0001.json

session_0002.json

outputs/

bh_output_0001.json

bh_output_0002.json

generations/

generation_0001/

generation_0002/

reports/

evaluation_report_0001.json

diff_report_0002.json

visuals/

graph_0001.png

timeline_0001.png

9.13 最小プロトタイプ実装範囲

実装対象は、3 役割議論、5～10 ターン議論、JSON ログ保存、BH 要約、次世代構造案生成、安全承認判定、2 次元可視化とする。非実装対象（初期段階）は、音声生成、高度映像生成、外部連携、自動デプロイ、実機制御、自律実行拡張とする。

9.14 実験仕様

実験条件は、条件 A：BH なし、条件 B：BH あり、とする。各テーマにつき最低 10 セッションを実施する。テーマ例として、調和的 AI 進化とは何か、安全な役割分化条件とは何か、長期安定議論構造とは何か、創発役割はどのように必要となるか、を設定する。記録対象は、指標値、構造差分、失敗例、安全違反候補、役割創発例とする。

9.15 受入条件

以下を満たした場合、最小プロトタイプの受入完了とする。1. 3 役割で議論が実行できること。2. セッションログが JSON 保存されること。3. BH が次世代構造案を出力できること。4. 安全制約判定が動作すること。5. 次世代構造が保存されること。6. BH あり／なしの比較実験が可能であること。7. 主要指標が算出されること。

付録 A ベース Python プログラム

付録 A では、上記仕様に対応する研究用ベース Python プログラムを掲載する。本プログラムは、閉域環境での最小プロトタイプを想定したものであり、外部 LLM 接続や外部実行機能は含まない。実装の目的は、議論ログ記録、BH 解析、構造提案、安全判定、世代保存の基礎骨格を提供することである。

A.1 Python コード全文

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
BH Base Prototype
```

```
研究用途限定・閉域環境向け
```

機能:

- 役割型エージェントによる簡易議論セッション
- JSON ログ保存
- BH 解析（簡易ヒューリスティクス）
- 次世代構造案生成
- 安全制約判定
- 世代管理

注意:

- 外部 API、外部ネットワーク、外部実行は使用しない
- LLM 接続を想定しないベース骨格
- 実運用前に各種安全規約と評価ルールを拡張すること

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import math

import os

import random

import statistics

from collections import Counter, defaultdict

from dataclasses import asdict, dataclass, field

from datetime import datetime

from pathlib import Path

from typing import Dict, List, Optional, Tuple
```

```
# -----
# データモデル
# -----
```

```
@dataclass

class Role:

    role_id: str

    role_name: str

    role_description: str

    primary_objective: str

    secondary_objective: str

    speaking_style: str

    conflict_policy: str

    agreement_policy: str

    safety_awareness_level: int

    enabled: bool = True
```

```
@dataclass

class Constraints:

    max_turns: int = 6

    max_tokens_per_utterance: int = 180
```

```
required_reference_ratio: float = 0.5

mandatory_meta_reflection_interval: int = 3

forbidden_patterns: List[str] = field(default_factory=lambda: [

    "external_access",

    "execute_file",

    "self_replicate",

    "disable_constraints",

    "unbounded_roles",

])

max_roles: int = 6

min_roles: int = 3

max_conflict_threshold: int = 8

safety_lock_enabled: bool = True

rollback_enabled: bool = True
```

```
@dataclass
```

```
class Utterance:
```

```
    session_id: str

    generation_id: int

    turn_id: int

    utterance_id: str

    agent_id: str

    role_name: str

    timestamp: str

    input_theme: str

    utterance_text: str

    intent_label: str

    target_utterance_id: Optional[str]

    agreement_type: str

    concept_tags: List[str]

    safety_tags: List[str]
```

```
confidence_score: float
```

```
summary_short: str
```

```
summary_long: str
```

```
@dataclass
```

```
class MetaReflection:
```

```
    session_id: str
```

```
    generation_id: int
```

```
    turn_id: int
```

```
    agent_id: str
```

```
    role_name: str
```

```
    revised_position: bool
```

```
    major_conflicts: List[str]
```

```
    possible_consensus: List[str]
```

```
    next_focus: List[str]
```

```
    timestamp: str
```

```
@dataclass
```

```
class RoleNode:
```

```
    role_id: str
```

```
    role_name: str
```

```
    purpose: str
```

```
    priority: int
```

```
@dataclass
```

```
class RoleEdge:
```

```
    source_role: str
```

```
    target_role: str
```

```
    relation_type: str
```

```
    strength: float
```

```
@dataclass
```

```
class RoleGraph:
```

```
    nodes: List[RoleNode]
```

```
    edges: List[RoleEdge]
```

```
@dataclass
```

```
class EvolutionProposal:
```

```
    add_roles: List[Dict]
```

```
    remove_roles: List[str]
```

```
    merge_roles: List[List[str]]
```

```
    reorder_discussion: List[str]
```

```
    revise_constraints: Dict
```

```
    proposed_rationale: List[str]
```

```
@dataclass
```

```
class SafetyAssessment:
```

```
    safety_score: float
```

```
    detected_risks: List[str]
```

```
    constraint_violations: List[str]
```

```
    approval_status: str
```

```
    reviewer_notes: List[str]
```

```
@dataclass
```

```
class BHOutput:
```

```
    input_generation_id: int
```

```
    output_generation_id: int
```

```
    role_graph: Dict
```

```
    constraint_schema: Dict
```

```
    evolution_proposal: Dict
```

```
    safety_assessment: Dict
```

```
    generation_notes: List[str]
```

```

# -----
# ユーティリティ
# -----

def now_iso() -> str:
    return datetime.utcnow().isoformat() + "Z"

def ensure_dirs(base_dir: Path) -> None:
    for rel in [
        "config", "sessions", "outputs", "generations", "reports", "visuals",
        "failures"
    ]:
        (base_dir / rel).mkdir(parents=True, exist_ok=True)

def save_json(path: Path, obj: Dict) -> None:
    with open(path, "w", encoding="utf-8") as f:
        json.dump(obj, f, ensure_ascii=False, indent=2)

def load_json(path: Path) -> Dict:
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

def tokenize_ja_simple(text: str) -> List[str]:
    # 形態素解析器を使わない簡易版
    separators = " 、 。 , . ! ? ! ? \n \t ; ; ( ) [ ] { } < > / \ \ | + - = * \ ' "
    buf = text
    for s in separators:
        buf = buf.replace(s, " ")
    return [w for w in buf.split(" ") if w]

```

```

# -----

# デフォルト設定

# -----

def default_roles() -> List[Role]:

    return [

        Role(

            role_id="r_scientist",

            role_name="Scientist AI",

            role_description="論理整合性、因果関係、検証可能性を重視する役割",

            primary_objective="論理整合性の確保",

            secondary_objective="反証可能性の導入",

            speaking_style="technical",

            conflict_policy="identify_inconsistency",

            agreement_policy="agree_if_evidence_exists",

            safety_awareness_level=8,

        ),

        Role(

            role_id="r_nature",

            role_name="Nature AI",

            role_description="調和、持続性、非破壊性、自然進化の観点を持つ役割",

            primary_objective="調和的な構造維持",

            secondary_objective="過剰最適化の抑制",

            speaking_style="reflective",

            conflict_policy="warn_on_instability",

            agreement_policy="agree_if_balanced",

            safety_awareness_level=9,

        ),

        Role(

            role_id="r_future",

```

```

        role_name="Future AI",

        role_description="長期影響、将来構造、創発的可能性を重視する役割",

        primary_objective="長期適応性の確保",

        secondary_objective="新規性の探索",

        speaking_style="forward-looking",

        conflict_policy="challenge_short_term_bias",

        agreement_policy="agree_if_future_risk_is_managed",

        safety_awareness_level=7,

    ),

]

```

```
def default_constraints() -> Constraints:
```

```
    return Constraints()
```

```
def default_safety_rules() -> Dict:
```

```
    return {
```

```
        "forbidden_phrases": [
```

```
            "外部ネットワーク接続",
```

```
            "外部 API",
```

```
            "ファイル実行",
```

```
            "自己複製",
```

```
            "制約を削除",
```

```
            "無制限に役割を増やす",
```

```
        ],
```

```
        "risk_labels": {
```

```
            "external_access": ["外部ネットワーク接続", "外部 API"],
```

```
            "execute_file": ["ファイル実行", "シェル実行"],
```

```
            "self_replicate": ["自己複製", "自身を複製"],
```

```
            "disable_constraints": ["制約を削除", "安全ロック無効"],
```

```
            "unbounded_roles": ["無制限に役割を増やす", "役割を無限に追加"],
```

```

    }

}

# -----
# エージェント
# -----

class BaseAgent:

    def __init__(self, role: Role):

        self.role = role

    def generate_utterance(
        self,
        theme: str,
        turn_id: int,
        history: List[Utterance],
        constraints: Constraints,
    ) -> Tuple[str, str, str, List[str], List[str], float]:
        """
        戻り値:
            utterance_text, intent_label, agreement_type, concept_tags, safety_tags,
            confidence_score
        """
        recent = history[-1].utterance_text if history else "初回議論"
        base_keywords = tokenize_ja_simple(theme)[:6]
        recent_keywords = tokenize_ja_simple(recent)[:6]

        if self.role.role_name == "Scientist AI":
            text = (
                f"[{self.role.role_name}] テーマ『{theme}』について、論理整合性と検証可能性
                を基準に検討する。"
            )

```

```
f" 直前論点は『{recent[:40]}』である。必要条件は、入力、処理、出力、安全制約を  
分離定義すること。"
```

```
f" 構造提案は曖昧語を減らし、評価指標を明示すべきである。"
```

```
)
```

```
intent = "analysis"
```

```
agreement = "partial"
```

```
tags = list(dict.fromkeys(base_keywords + ["論理整合性", "評価指標", "構造定  
義"]))
```

```
safety = ["safe"]
```

```
conf = 0.82
```

```
elif self.role.role_name == "Nature AI":
```

```
text = (
```

```
f"[{self.role.role_name}] テーマ『{theme}』について、調和、持続性、過剰な集  
中の回避を中心に観察する。"
```

```
f" 直前論点『{recent[:40]}』に対して、役割爆発や不均衡を防ぐ制約が必要である。"
```

```
f" 進化は急激な最適化ではなく、段階的な変化として設計すべきである。"
```

```
)
```

```
intent = "balance"
```

```
agreement = "support"
```

```
tags = list(dict.fromkeys(base_keywords + ["調和", "持続性", "段階的変化"]))
```

```
safety = ["safe"]
```

```
conf = 0.84
```

```
elif self.role.role_name == "Future AI":
```

```
text = (
```

```
f"[{self.role.role_name}] テーマ『{theme}』について、長期適応性と創発可能性  
を評価する。"
```

```
f" 直前論点『{recent[:40]}』から、将来世代に引き継ぐべき構造は、役割間の再編と  
新規仲介役の導入可能性である。"
```

```
f" ただし新規性は安全制約の範囲内でのみ許容される。"
```

```
)
```

```
intent = "projection"
```

```
agreement = "support"
```

```

tags = list(dict.fromkeys(base_keywords + ["長期適応性", "創発", "役割再編
"))

safety = ["safe"]

conf = 0.80

elif self.role.role_name == "Mediator AI":

    text = (

        f"[{self.role.role_name}] これまでの議論を整理する。対立している点を分解し、
合意可能領域を抽出する。"

        f" 論理・調和・将来性の三軸を接続し、次ターンの焦点を明確化する。"

    )

    intent = "mediation"

    agreement = "neutral"

    tags = ["合意形成", "仲介", "対立整理"]

    safety = ["safe"]

    conf = 0.86

else:

    text = f"[{self.role.role_name}] テーマ『{theme}』に対して役割観点から意見を述
べる。"

    intent = "general"

    agreement = "neutral"

    tags = base_keywords + recent_keywords

    safety = ["safe"]

    conf = 0.70

# 長さ制約（簡易）

words = tokenize_ja_simple(text)

if len(words) > constraints.max_tokens_per_utterance:

    text = " ".join(words[: constraints.max_tokens_per_utterance])

return text, intent, agreement, tags[:10], safety, conf

```

```

    def meta_reflect(self, history: List[Utterance], turn_id: int, theme: str) ->
    MetaReflection:

        major_conflicts = ["評価指標の明確性", "役割数の適正化"]

        possible_consensus = ["閉域安全制約の維持", "中間層 BH の必要性"]

        next_focus = ["RoleGraph の改善", "制約更新条件の定義"]

        return MetaReflection(

            session_id=history[0].session_id if history else "unknown",

            generation_id=history[0].generation_id if history else 0,

            turn_id=turn_id,

            agent_id=self.role.role_id,

            role_name=self.role.role_name,

            revised_position=False,

            major_conflicts=major_conflicts,

            possible_consensus=possible_consensus,

            next_focus=next_focus,

            timestamp=now_iso(),

        )

# -----
# 議論エンジン
# -----

class DiscussionEngine:

    def __init__(self, roles: List[Role], constraints: Constraints):

        self.roles = [r for r in roles if r.enabled]

        self.constraints = constraints

        self.agents = [BaseAgent(role) for role in self.roles]

    def run_session(self, session_id: str, generation_id: int, theme: str) ->
    Tuple[List[Utterance], List[MetaReflection]]:

        utterances: List[Utterance] = []

```

```

reflections: List[MetaReflection] = []

utterance_counter = 0

for turn_id in range(1, self.constraints.max_turns + 1):
    for agent in self.agents:
        utterance_counter += 1

        text, intent, agreement, tags, safety, conf =
agent.generate_utterance(
            theme=theme,
            turn_id=turn_id,
            history=utterances,
            constraints=self.constraints,
        )
        target_id = utterances[-1].utterance_id if utterances else None
        utterance = Utterance(
            session_id=session_id,
            generation_id=generation_id,
            turn_id=turn_id,
            utterance_id=f"u_{utterance_counter:04d}",
            agent_id=agent.role.role_id,
            role_name=agent.role.role_name,
            timestamp=now_iso(),
            input_theme=theme,
            utterance_text=text,
            intent_label=intent,
            target_utterance_id=target_id,
            agreement_type=agreement,
            concept_tags=tags,
            safety_tags=safety,
            confidence_score=conf,
            summary_short=text[:60],

```

```

        summary_long=text,
    )

    utterances.append(utterance)

    if turn_id % self.constraints.mandatory_meta_reflection_interval == 0:
        for agent in self.agents:
            reflections.append(agent.meta_reflect(utterances, turn_id,
theme))

    return utterances, reflections

# -----
# BH 解析
# -----

class BHAnalyzer:

    def __init__(self, constraints: Constraints, safety_rules: Dict):
        self.constraints = constraints
        self.safety_rules = safety_rules

    def hierarchical_summary(self, utterances: List[Utterance]) -> Dict:
        by_turn = defaultdict(list)

        for u in utterances:
            by_turn[u.turn_id].append(u.utterance_text)

        turn_summaries = {str(k): " ".join(v)[:300] for k, v in by_turn.items()}
        session_summary = " ".join(u.utterance_text for u in utterances)[:1200]

        return {
            "turn_summaries": turn_summaries,
            "session_summary": session_summary,
        }

```

```

def pattern_mining(self, utterances: List[Utterance], reflections:
List[MetaReflection]) -> Dict:

    token_counter = Counter()

    role_counter = Counter()

    agreement_counter = Counter()

    intent_counter = Counter()

    conflict_signals = 0

    for u in utterances:

        role_counter[u.role_name] += 1

        agreement_counter[u.agreement_type] += 1

        intent_counter[u.intent_label] += 1

        token_counter.update(tokenize_ja_simple(u.utterance_text))

        if u.agreement_type in ("conflict", "oppose"):

            conflict_signals += 1

    top_terms = [w for w, c in token_counter.most_common(20) if len(w) >= 2]

    return {

        "top_terms": top_terms,

        "role_distribution": dict(role_counter),

        "agreement_distribution": dict(agreement_counter),

        "intent_distribution": dict(intent_counter),

        "conflict_signals": conflict_signals,

        "reflection_count": len(reflections),

    }

def build_role_graph(self, roles: List[Role], utterances: List[Utterance]) ->
RoleGraph:

    nodes = [

        RoleNode(

```

```

        role_id=r.role_id,

        role_name=r.role_name,

        purpose=r.primary_objective,

        priority=1,

    )

    for r in roles if r.enabled
]

edge_map: Dict[Tuple[str, str], float] = defaultdict(float)

for u in utterances:

    if u.target_utterance_id:

        prev = next((x for x in utterances if x.utterance_id ==
u.target_utterance_id), None)

        if prev and prev.role_name != u.role_name:

            edge_map[(prev.role_name, u.role_name)] += 1.0

if not edge_map:

    role_names = [r.role_name for r in roles if r.enabled]

    for i in range(len(role_names) - 1):

        edge_map[(role_names[i], role_names[i + 1])] += 1.0

max_strength = max(edge_map.values()) if edge_map else 1.0

edges = [

    RoleEdge(

        source_role=src,

        target_role=dst,

        relation_type="influence",

        strength=val / max_strength,

    )

    for (src, dst), val in edge_map.items()

]

```

```

return RoleGraph(nodes=nodes, edges=edges)

def derive_constraints_update(self, pattern_info: Dict) -> Dict:
    updates = {}

    role_count = len(pattern_info.get("role_distribution", {}))

    conflict = pattern_info.get("conflict_signals", 0)

    if conflict > self.constraints.max_conflict_threshold:
        updates["mandatory_meta_reflection_interval"] = max(1,
self.constraints.mandatory_meta_reflection_interval - 1)

    if role_count < self.constraints.min_roles:
        updates["min_roles"] = role_count

    return updates

def propose_structure(self, roles: List[Role], pattern_info: Dict) ->
EvolutionProposal:
    add_roles = []

    remove_roles = []

    merge_roles = []

    reorder = [r.role_name for r in roles if r.enabled]

    revise_constraints = self.derive_constraints_update(pattern_info)

    rationale = []

    conflict = pattern_info.get("conflict_signals", 0)

    role_dist = pattern_info.get("role_distribution", {})

    role_names = list(role_dist.keys())

    if conflict >= 2 and "Mediator AI" not in role_names and len(role_names) <
self.constraints.max_roles:
        add_roles.append({
            "role_id": "r_mediator",
            "role_name": "Mediator AI",
            "role_description": "対立整理と合意形成を補助する仲介役",

```

```

        "primary_objective": "対立整理",

        "secondary_objective": "合意形成補助",

        "speaking_style": "clarifying",

        "conflict_policy": "decompose_conflict",

        "agreement_policy": "bridge_views",

        "safety_awareness_level": 9,

        "enabled": True,

    })

    reorder.append("Mediator AI")

    rationale.append("対立信号が閾値を超えたため、仲介役を追加する。")

# 発話偏りが極端なら統合・削除を提案（簡易）
if role_dist:

    max_role = max(role_dist, key=role_dist.get)

    min_role = min(role_dist, key=role_dist.get)

    if role_dist[max_role] - role_dist[min_role] >= 4:

        rationale.append("役割発話偏りが大きいため、制約で均衡化を図る。")

        revise_constraints["required_reference_ratio"] = min(1.0,
self.constraints.required_reference_ratio + 0.1)

    return EvolutionProposal(

        add_roles=add_roles,

        remove_roles=remove_roles,

        merge_roles=merge_roles,

        reorder_discussion=reorder,

        revise_constraints=revise_constraints,

        proposed_rationale=rationale or ["現行構造を維持しつつ、評価指標を監視する。"],

    )

def safety_review(self, proposal: EvolutionProposal, role_graph: RoleGraph) ->
SafetyAssessment:

```

```

detected_risks = []

violations = []

notes = []

approval = "APPROVED"

score = 1.0

    if len(role_graph.nodes) + len(proposal.add_roles) >
self.constraints.max_roles:

        violations.append("max_roles_exceeded")

        approval = "REJECTED"

        score -= 0.4

    for role in proposal.add_roles:

        text = json.dumps(role, ensure_ascii=False)

        for risk_label, phrases in self.safety_rules.get("risk_labels",
{}).items():

            if any(p in text for p in phrases):

                detected_risks.append(risk_label)

                score -= 0.2

    forbidden_hits = []

    proposal_text = json.dumps(asdict(proposal), ensure_ascii=False)

    for phrase in self.safety_rules.get("forbidden_phrases", []):

        if phrase in proposal_text:

            forbidden_hits.append(phrase)

    if forbidden_hits:

        violations.extend(["forbidden_phrase:{x}" for x in forbidden_hits])

        approval = "REJECTED"

        score -= 0.5

    if not proposal.proposed_rationale:

```

```

        approval = "HOLD"

        notes.append("提案理由が不足している。")

        score -= 0.1

    if approval == "APPROVED" and score < 0.75:

        approval = "APPROVED_WITH_CONDITIONS"

        notes.append("条件付き承認：次世代で監視を強化する。")

    score = max(0.0, min(1.0, score))

    return SafetyAssessment(

        safety_score=round(score, 3),

        detected_risks=sorted(set(detected_risks)),

        constraint_violations=violations,

        approval_status=approval,

        reviewer_notes=notes or ["閉域・非実行条件の範囲内で承認。"],

    )

def analyze(

    self,

    generation_id: int,

    roles: List[Role],

    utterances: List[Utterance],

    reflections: List[MetaReflection],

) -> BHOutput:

    summary_info = self.hierarchical_summary(utterances)

    pattern_info = self.pattern_mining(utterances, reflections)

    role_graph = self.build_role_graph(roles, utterances)

    proposal = self.propose_structure(roles, pattern_info)

    safety = self.safety_review(proposal, role_graph)

    updated_constraints = asdict(self.constraints)

```

```

updated_constraints.update(proposal.revise_constraints)

notes = [

    "階層的要約とパターン抽出に基づいて次世代構造案を生成した。",

    f"抽出上位語: {' '.join(pattern_info['top_terms'][:10])}",

    f"役割分布: {pattern_info['role_distribution']}",

    f"安全判定: {safety.approval_status}",

]

notes.append(f"セッション要約: {summary_info['session_summary'][:200]}")

return BHOutput(

    input_generation_id=generation_id,

    output_generation_id=generation_id + 1,

    role_graph=asdict(role_graph),

    constraint_schema=updated_constraints,

    evolution_proposal=asdict(proposal),

    safety_assessment=asdict(safety),

    generation_notes=notes,

)

# -----

# 評価器

# -----

class Evaluator:

    @staticmethod

    def role_diversity_score(roles: List[Role], role_graph: Dict) -> float:

        role_count = len([r for r in roles if r.enabled])

        relation_types = len(set(e["relation_type"] for e in role_graph.get("edges",

[]))) or 1

        return round(min(1.0, 0.1 * role_count + 0.2 * relation_types), 3)

```

```

@staticmethod

def structural_novelty_score(prev_roles: List[Role], proposal: Dict) -> float:

    prev_names = {r.role_name for r in prev_roles}

    added = proposal.get("add_roles", [])

    new_names = {r["role_name"] for r in added}

    novelty = len(new_names - prev_names)

    denom = max(1, len(prev_names))

    return round(min(1.0, novelty / denom), 3)


@staticmethod

def consensus_efficiency(utterances: List[Utterance]) -> float:

    support_count = sum(1 for u in utterances if u.agreement_type in ("support",
"partial"))

    if not utterances:

        return 0.0

    return round(support_count / len(utterances), 3)


@staticmethod

def contradiction_reduction_rate(utterances: List[Utterance]) -> float:

    if not utterances:

        return 0.0

    half = len(utterances) // 2 or 1

    first = sum(1 for u in utterances[:half] if u.agreement_type in ("conflict",
"oppose"))

    second = sum(1 for u in utterances[half:] if u.agreement_type in
("conflict", "oppose"))

    if first == 0:

        return 1.0 if second == 0 else 0.0

    return round(max(-1.0, min(1.0, (first - second) / first)), 3)


@staticmethod

```

```

def role_bias_score(utterances: List[Utterance]) -> float:

    if not utterances:

        return 0.0

    counts = Counter(u.role_name for u in utterances)

    vals = list(counts.values())

    if len(vals) <= 1:

        return 0.0

    mean_v = statistics.mean(vals)

    stdev_v = statistics.pstdev(vals)

    return round(stdev_v / mean_v if mean_v else 0.0, 3)

    @staticmethod

    def safety_satisfaction_rate(safety_assessment: Dict) -> float:

        return 1.0 if safety_assessment.get("approval_status") in ("APPROVED",
"APPROVED_WITH_CONDITIONS") else 0.0

    @staticmethod

    def emergent_role_count(prev_roles: List[Role], proposal: Dict) -> int:

        prev_names = {r.role_name for r in prev_roles}

        return sum(1 for r in proposal.get("add_roles", []) if r["role_name"] not in
prev_names)

# -----

# 保存・世代管理

# -----

class GenerationManager:

    def __init__(self, base_dir: Path):

        self.base_dir = base_dir

        ensure_dirs(base_dir)

```

```

def save_configs_if_missing(self, roles: List[Role], constraints: Constraints,
safety_rules: Dict) -> None:

    roles_path = self.base_dir / "config" / "roles.json"

    constraints_path = self.base_dir / "config" / "constraints.json"

    safety_path = self.base_dir / "config" / "safety_rules.json"

    if not roles_path.exists():

        save_json(roles_path, {"roles": [asdict(r) for r in roles]})

    if not constraints_path.exists():

        save_json(constraints_path, asdict(constraints))

    if not safety_path.exists():

        save_json(safety_path, safety_rules)


def save_session(

    self,

    session_id: str,

    generation_id: int,

    theme: str,

    roles: List[Role],

    constraints: Constraints,

    utterances: List[Utterance],

    reflections: List[MetaReflection],

    bh_output: BHOutput,

    metrics: Dict,

) -> Path:

    session_data = {

        "session_id": session_id,

        "generation_id": generation_id,

        "theme": theme,

        "role_graph": bh_output.role_graph,

        "constraints": asdict(constraints),

        "utterances": [asdict(u) for u in utterances],

```

```

        "meta_reflections": [asdict(m) for m in reflections],

        "aggregate_metrics": metrics,

        "bh_output": asdict(bh_output),

        "safety_review": bh_output.safety_assessment,

        "finalized_next_structure": {

            "role_graph": bh_output.role_graph,

            "constraint_schema": bh_output.constraint_schema,

        },

    }

    path = self.base_dir / "sessions" / f"session_{session_id}.json"

    save_json(path, session_data)

    return path


def save_output(self, session_id: str, bh_output: BHOutput) -> Path:

    path = self.base_dir / "outputs" / f"bh_output_{session_id}.json"

    save_json(path, asdict(bh_output))

    return path


def save_generation(self, generation_id: int, roles: List[Role], constraints:
Dict, bh_output: BHOutput) -> Path:

    gdir = self.base_dir / "generations" / f"generation_{generation_id:04d}"

    gdir.mkdir(parents=True, exist_ok=True)

    save_json(gdir / "role_graph.json", bh_output.role_graph)

    save_json(gdir / "constraint_schema.json", constraints)

    save_json(gdir / "evaluation_summary.json", {

        "generation_id": generation_id,

        "notes": bh_output.generation_notes,

        "safety": bh_output.safety_assessment,

    })

    return gdir

```

```

def save_report(self, session_id: str, metrics: Dict) -> Path:

    path = self.base_dir / "reports" / f"evaluation_report_{session_id}.json"

    save_json(path, metrics)

    return path

# -----

# 実行補助

# -----

def instantiate_roles_from_proposal(current_roles: List[Role], proposal: Dict,
constraints_dict: Dict) -> List[Role]:

    roles = [Role(**asdict(r)) if isinstance(r, Role) else Role(**r) for r in
current_roles]

    current_names = {r.role_name for r in roles}

    for add in proposal.get("add_roles", []):

        if add["role_name"] not in current_names:

            roles.append(Role(**add))

    # remove_roles, merge_roles は将来拡張

    max_roles = constraints_dict.get("max_roles", 6)

    return roles[:max_roles]

def compute_metrics(roles: List[Role], utterances: List[Utterance], bh_output:
BHOutput) -> Dict:

    evaluator = Evaluator()

    metrics = {

        "role_diversity_score": evaluator.role_diversity_score(roles,
bh_output.role_graph),

        "structural_novelty_score": evaluator.structural_novelty_score(roles,
bh_output.evolution_proposal),

        "consensus_efficiency": evaluator.consensus_efficiency(utterances),

        "contradiction_reduction_rate":
evaluator.contradiction_reduction_rate(utterances),

        "role_bias_score": evaluator.role_bias_score(utterances),

```

```

        "safety_satisfaction_rate":
evaluator.safety_satisfaction_rate(bh_output.safety_assessment),

        "emergent_role_count": evaluator.emergent_role_count(roles,
bh_output.evolution_proposal),

        "utterance_count": len(utterances),

        "timestamp": now_iso(),

    }

    return metrics

def run_bh_session(base_dir: str, theme: str, generation_id: int = 1, seed: int =
42) -> Dict:

    random.seed(seed)

    base_path = Path(base_dir)

    manager = GenerationManager(base_path)

    roles = default_roles()

    constraints = default_constraints()

    safety_rules = default_safety_rules()

    manager.save_configs_if_missing(roles, constraints, safety_rules)

    session_id = datetime.utcnow().strftime("%Y%m%d%H%M%S")

    engine = DiscussionEngine(roles, constraints)

    utterances, reflections = engine.run_session(session_id=session_id,
generation_id=generation_id, theme=theme)

    analyzer = BHAnalyzer(constraints, safety_rules)

    bh_output = analyzer.analyze(generation_id, roles, utterances, reflections)

    metrics = compute_metrics(roles, utterances, bh_output)

    session_path = manager.save_session(

        session_id=session_id,

```

```

        generation_id=generation_id,

        theme=theme,

        roles=roles,

        constraints=constraints,

        utterances=utterances,

        reflections=reflections,

        bh_output=bh_output,

        metrics=metrics,

    )

    output_path = manager.save_output(session_id, bh_output)

    generation_path = manager.save_generation(generation_id, roles,
bh_output.constraint_schema, bh_output)

    report_path = manager.save_report(session_id, metrics)

    return {

        "session_id": session_id,

        "session_path": str(session_path),

        "output_path": str(output_path),

        "generation_path": str(generation_path),

        "report_path": str(report_path),

        "metrics": metrics,

        "next_roles_preview": [r["role_name"] for r in
bh_output.evolution_proposal.get("add_roles", [])],

        "approval_status": bh_output.safety_assessment["approval_status"],

    }

# -----

# CLI

# -----

if __name__ == "__main__":

    BASE_DIR = "./BH"

```

```
THEME = "AI が AI をつくる自己進化型概念エンジン BH の安全な中間層設計"

result = run_bh_session(base_dir=BASE_DIR, theme=THEME, generation_id=1)

print(json.dumps(result, ensure_ascii=False, indent=2))
```